Программное обеспечение для определения направления магнитных моментов на основе магнитооптического эффекта Керра

Студент:
Моисеев Антон Октаевич
группа 419

Руководители:
Перов Николай Сергеевич
Голубцов Петр Викторович
Перова Наталья Николаевна

Москва — 2025

Содержание

1 Введение **3**

2 Магнитооптический эффект Керра (МОЭК) и принципы работы микроскопа Керра **3**
2.1 Физические основы МОЭК и тензор диэлектрической проницаемости . 3
2.1.1 Тензор диэлектрической проницаемости 3
2.1.2 Комплексный угол Керра 4
2.1.3 Геометрии МОЭК 4
2.2 Ключевые принципы работы микроскопа Керра и количественный анализ 4
2.2.1 Интенсивность и создание контраста 4
2.2.2 Векторная и количественная визуализация 5
2.2.3 Процедура количественной реконструкции 5

3 Постановка задачи **6**

4 Основная часть **7**
4.1 Определения границ доменов 7

5 Выводы **10**

A Физическое приложение **12**
A.1 Магнитооптический эффект Керра 12
A.2 Геометрии и чувствительность 12
A.3 Магнитные домены и стенки 12
A.4 Линейная модель сигнала и ограничения 12

B Математическое приложение **13**
B.1 Постановка 13
B.2 Нормировка и наблюдаемые 13
B.3 Линейная модель и один кадр 13
B.4 Два независимых кадра 13
B.5 Сегментация доменов 14
B.6 Оценка локальной ориентации: структурный тензор 14
B.7 Предобработка и устойчивость 14
B.8 Калибровка на насыщении и знак 14
B.9 Проверка независимости и ковариация 14
B.10 Псевдокод конвейера 15

C Программное приложение **15**
C.1 Архитектура приложения 15
C.2 Алгоритмы обработки и анализа 15
C.2.1 Предобработка и кэширование 15
C.2.2 Автоматическая пороговая сегментация (Метод Оцу) 16
C.2.3 Сегментация и фильтрация доменов 16
C.2.4 Построение векторной диаграммы 16
C.3 Пользовательский интерфейс 16

D Листинг программного кода **18**
D.1 Точка входа (course4_programApp.swift) 18
D.2 Интерфейс пользователя (ContentView.swift) 18
D.3 Логика анализа (generateVectorDiagram.swift) 23

1 Введение

Магнитооптическая микроскопия по эффекту Керра позволяет визуализировать магнитные домены и их динамику за счёт зависимости поляризационных характеристик отражённого света от вектора намагниченности в зоне зондирования. В тонких плёнках и мультиматериалах эта техника даёт прямую чувствительность к определённой проекции намагниченности, что делает возможным извлечение направления магнитного момента по одиночному или паре изображений.

Цель работы — разработать и верифицировать алгоритм и программный прототип на Swift/macOS для оценки направления магнитного момента по МОЭК-изображениям с учётом калибровок, устойчивости и метрик качества, а также сформировать целостное описание физической, математической и программной частей.

Данное ПО, разработанное на основе физических законов и классического машинного зрения, не требует серверных мощностей и может быть применимо для анализа различных доменных структур даже на мобильных устройствах, что значительно упрощает и ускоряет анализ различных материалов.

2 Магнитооптический эффект Керра (МОЭК) и принципы работы микроскопа Керра

Магнитооптический эффект Керра (МОЭК), обнаруженный Джоном Керром в 1876 году, является основным явлением, используемым в магнитооптической микроскопии. МОЭК описывает изменение поляризации (вращение θ_K и эллиптичность ε_K) и/или интенсивности света при его отражении от намагниченной поверхности.

2.1 Физические основы МОЭК и тензор диэлектрической проницаемости

Магнитооптические эффекты, такие как МОЭК и вращение Фарадея, имеют свое происхождение в спин-орбитальном и обменном взаимодействиях. В первом порядке эти эффекты линейно зависят от намагниченности $\mathbf{M}$.

2.1.1 Тензор диэлектрической проницаемости

МОЭК возникает из-за того, что намагниченность $\mathbf{M}$ делает материал оптически анизотропным. Теория магнитооптики требует рассмотрения тензора диэлектрической проницаемости $\hat{\varepsilon}$, который в присутствии намагниченности принимает недиагональные компоненты. Для изотропных или кубических материалов этот тензор в линейном приближении относительно намагниченности $\mathbf{M} = (m_x, m_y, m_z)$ имеет вид:

$$\hat{\varepsilon} = \varepsilon_0 \begin{pmatrix} 1 & -iQm_z & iQm_y \\ iQm_z & 1 & -iQm_x \\ -iQm_y & iQm_x & 1 \end{pmatrix}$$

где ε_0 — диэлектрическая проницаемость немагнитного материала, а Q — магнитооптический параметр гирации (или параметр Керра), который является комплексной величиной и определяет силу эффекта.

2.1.2 Комплексный угол Керра

Изменение поляризации отражённого света количественно описывается комплексным углом Керра Ψ_K, который включает поворот (rotation) θ_K и эллиптичность (ellipticity) ε_K:

$$\Psi_K = \theta_K + i\varepsilon_K$$

Оба этих параметра, θ_K и ε_K, пропорциональны намагниченности, а также зависят от длины волны света (дисперсии) и оптических констант материала.

2.1.3 Геометрии МОЭК

Наблюдаемый эффект зависит от взаимного расположения вектора намагниченности **M** и плоскости падения света, что приводит к трём основным геометриям:

1. **Полярный эффект Керра (P-MOKE):** Намагниченность **M** перпендикулярна поверхности образца ($m_z \neq 0$). Обычно свет падает нормально или почти нормально.
2. **Продольный эффект Керра (L-MOKE):** Намагниченность **M** лежит в плоскости образца и параллельна плоскости падения света.
3. **Поперечный эффект Керра (T-MOKE):** Намагниченность **M** лежит в плоскости образца и перпендикулярна плоскости падения света. T-MOKE уникален тем, что не вызывает вращения поляризации, а приводит к изменению коэффициента отражения (модуляции интенсивности).

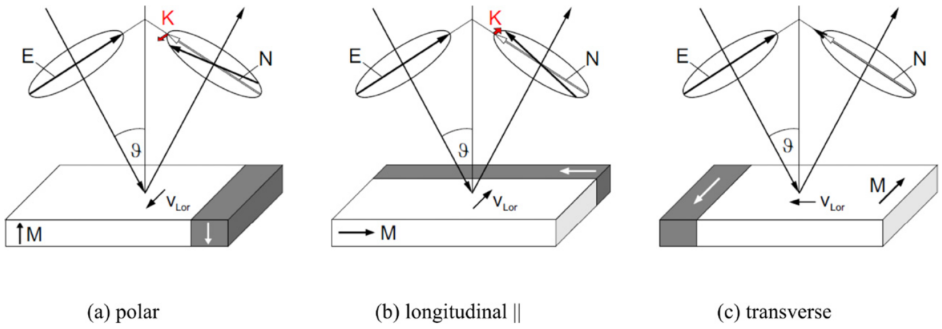

Рис. 1: Схематическое изображение конфигураций полярного, продольного и поперечного эффектов Керра, иллюстрирующих ориентацию вектора намагниченности **M** относительно падающего света.

2.2 Ключевые принципы работы микроскопа Керра и количественный анализ

МОЭК-микроскопия обычно использует поляризационный оптический микроскоп, где поляризатор создает линейно поляризованный падающий свет, а анализатор преобразует изменение поляризации отраженного света в контраст интенсивности.

2.2.1 Интенсивность и создание контраста

В широкопольной (wide-field) микроскопии Керра для создания контраста анализатор устанавливается в положение, близкое к скрещенному (почти гашение света).

Для количественной МОЭК-микроскопии отраженная интенсивность I может быть аппроксимирована как линейная функция компонент намагниченности в плоскости образца:

$$I = I_0 + Bm_l + Cm_t + D \quad (1)$$

где:

- I_0 — фоновая интенсивность, не зависящая от намагниченности.
- m_l и m_t — нормированные продольная и поперечная компоненты намагниченности (вдоль и перпендикулярно плоскости падения).
- B и C — коэффициенты, зависящие от нормальных (N) и керровских (K) амплитуд.
- D — член второго порядка по намагниченности (квадратичный), порядка K^2.

В классическом подходе для наблюдения доменов, не зависящий от намагниченности вклад I_0 устраняется с помощью метода разностного изображения. При этом член D (квадратичный по намагниченности) можно пренебречь, если анализатор и поляризатор настроены таким образом, что N велик по сравнению с K.

2.2.2 Векторная и количественная визуализация

Для полного определения векторного поля намагниченности $\mathbf{M} = (m_x, m_y, m_z)$ в плоскости (m_x, m_y) необходимо получить два изображения с ортогональными чувствительностями. В современных микроскопах Керра это достигается путем:

1. **Управления направлением падения:** Используется специальный источник света с массивом из восьми светодиодов, свет которых направляется через оптические волокна. Включение соответствующих светодиодов позволяет устанавливать направление падения света (и, следовательно, направление чувствительности Керра) без механических перемещений. Направление чувствительности (плоскость падения) определяется расположением источника света (концов волокон) в апертурной плоскости объектива.
2. **Импульсный режим и вычитание:** Изображения записываются в импульсном режиме с поочередным включением противоположных светодиодов. Вычитание двух изображений, полученных при противоположных направлениях падения света, позволяет получить чистый плоскостной контраст Керра.
3. **Устранение паразитного эффекта Фарадея:** Этот импульсный режим с вычитанием также эффективно устраняет паразитные вращения Фарадея, которые возникают в линзе объектива под действием внешнего магнитного поля и искажают сигнал МОЭК.

2.2.3 Процедура количественной реконструкции

Количественный анализ требует регистрации двух синусоидальных калибровочных функций, сдвинутых по фазе на 90°, что обеспечивает два линейных уравнения для двух искомых компонент намагниченности m_l и m_t.

В продвинутой методике, разработанной Soldatov и Schäfer, для устранения паразитных вкладов Фарадея и получения чистого контраста, используется разность интенсивностей I_{diff}:

$$I_{\text{diff}} = I(H) - I(-H)$$

где $I(H)$ и $I(-H)$ — интенсивности, измеренные при противоположных магнитных полях (или противоположных намагниченных состояниях).

Для реконструкции векторного поля намагниченности **M**, количественная микроскопия Керра основывается на сравнении интенсивности каждого пикселя, измеренной при двух ортогональных чувствительностях, с локальными калибровочными функциями.

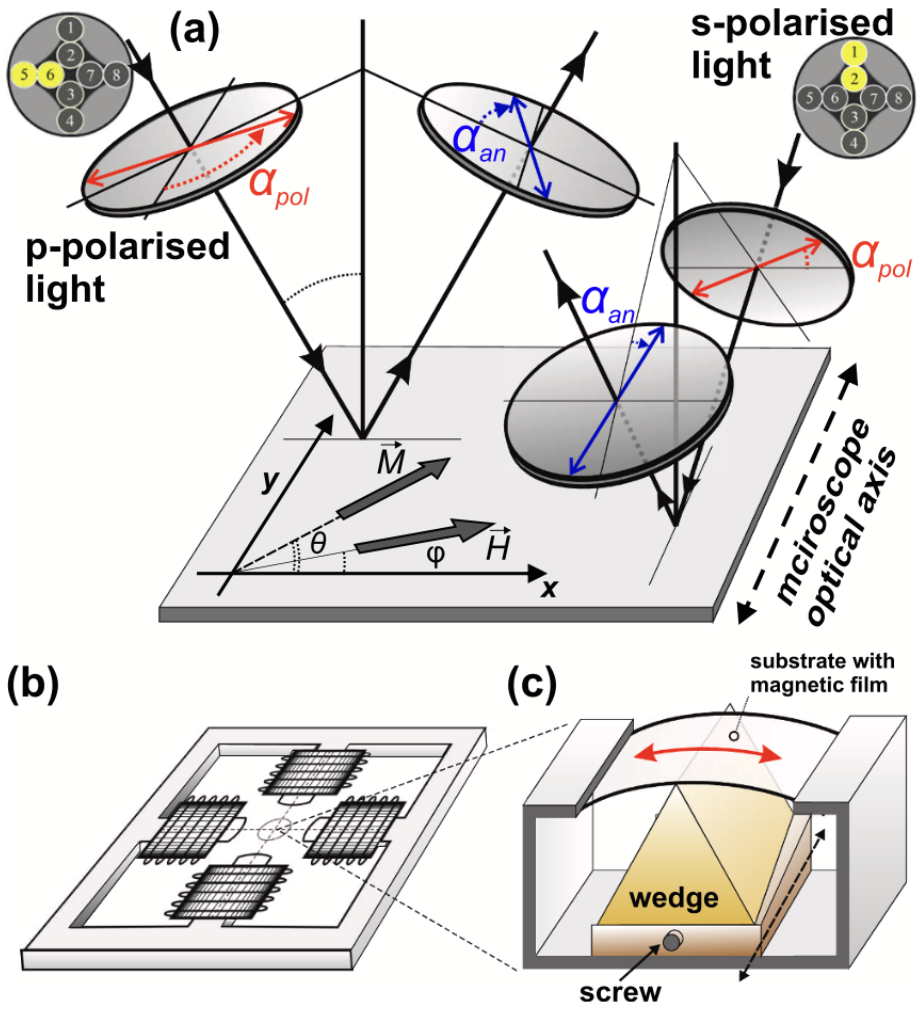

Рис. 2: Схематическое изображение установки широкопольного МОЭК-магнитометра, включающего массив светодиодов (LED cross), управляемых компьютером, для установки направления падения света и обеспечения ортогональных чувствительностей. (Источник: Рисунок 1(a) из "Advanced, Kerr-microscopy-based MOKE magnetometry for the anisotropy characterisation of magnetic films").

3 Постановка задачи

Физическая часть:

- Аналитическое решение задачи.
- И тд.

Программная часть:

- Реализация простого кроссплатформенного UI.
- Определения границ доменов с помощью машинного зрения.

- Программное реализация. физических законов.
- И тд.

4 Основная часть

4.1 Определения границ доменов

Для анализа изображений магнитных доменов в образцах использовалась разработанная программная реализация метода, основанного на вычислении яркости пикселей и определении светлых и тёмных областей, соответствующих границам доменов.

Анализ начинается с получения матрицы яркости каждого пикселя изображения. Яркость вычисляется по формуле преобразования RGB-компонентов в яркостное значение:

$$Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

Далее для каждого пикселя проводится классификация на основе пороговых значений яркости, выявляются светлые и тёмные границы доменов. Определяются ключевые точки границ и вычисляются центры доменов, а также векторные поля направлений на основе локальных свойств изображения.

Для визуализации результатов представлены две типовые микроскопические фотографии: исходная (`figdom1`) и та же фотография с нанесёнными границами доменов (`figdom2`).

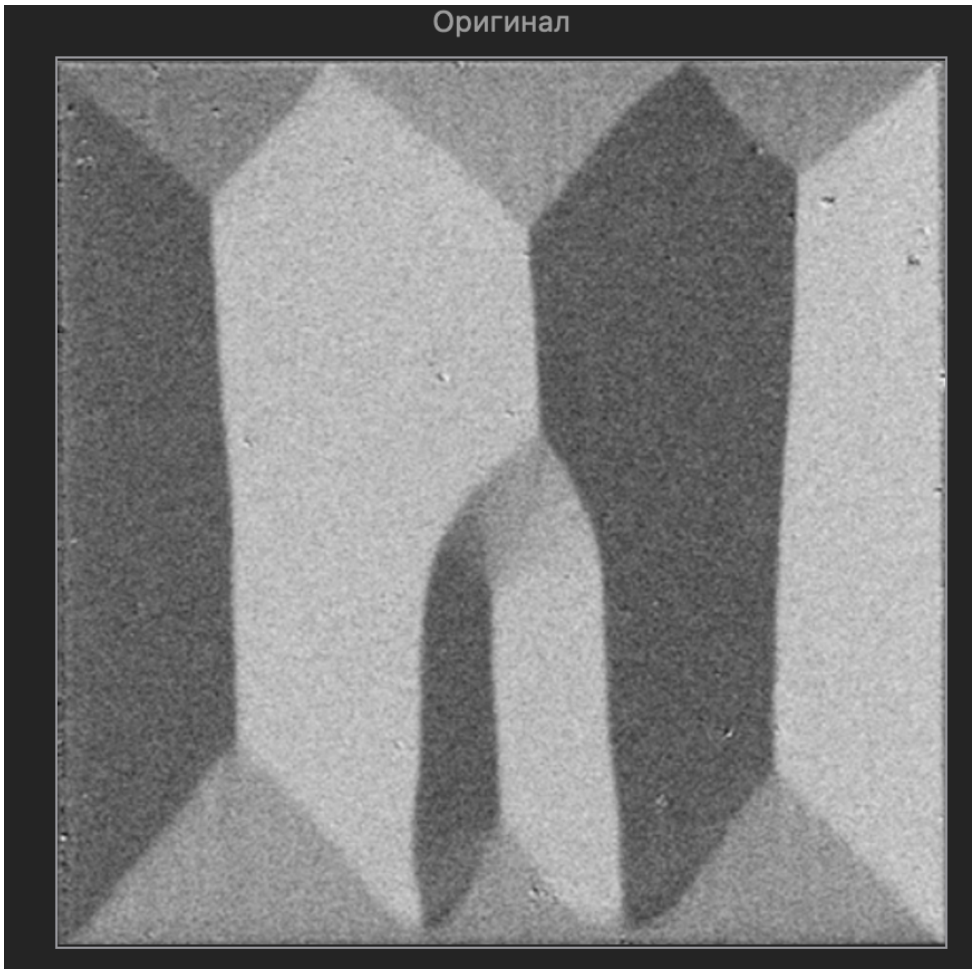

Рис. 3: Исходное микроскопическое изображение магнитных доменов.

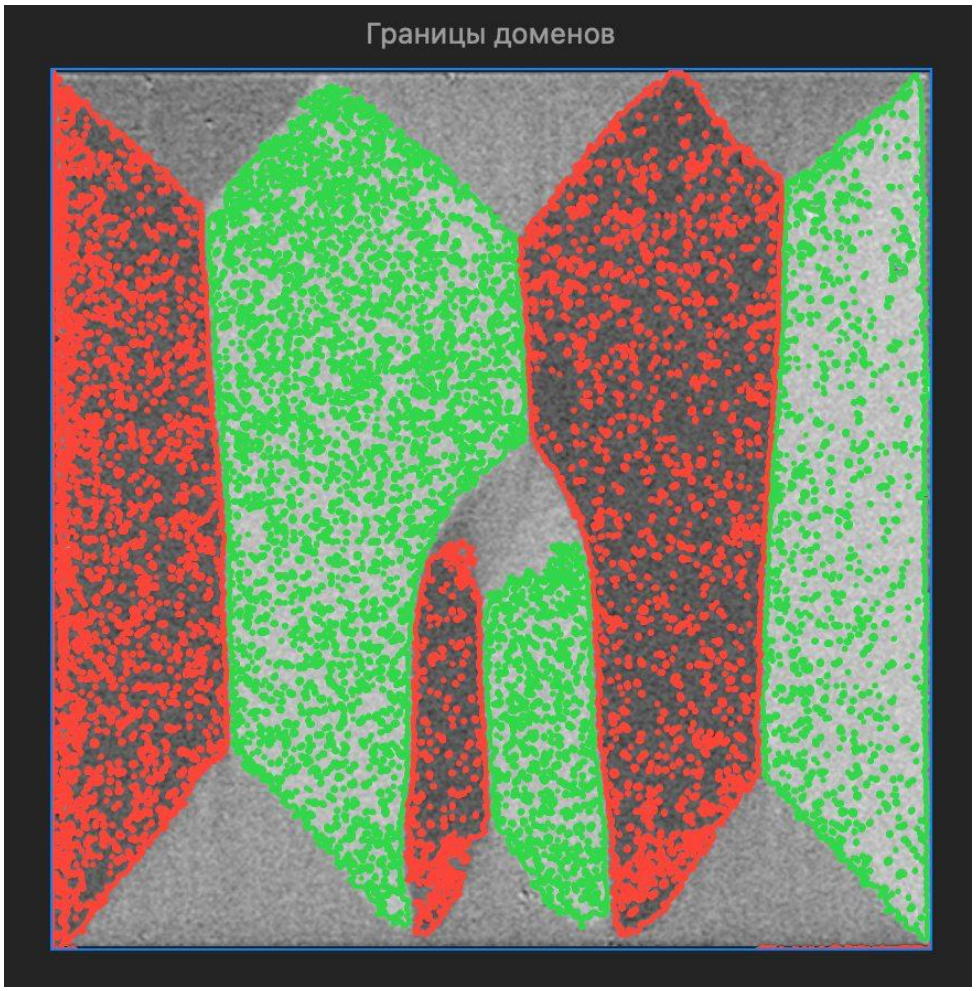

Рис. 4: Микроскопическое изображение с наложенными границами доменов, выделенными программным методом.

Кратко о физической модели

МОКЭ проявляется как вращение плоскости поляризации и эллиптичность отражённого света, линейно зависящие (в первом приближении) от соответствующей проекции $\mathbf{M}$ на вектор чувствительности $\mathbf{b}$, определяемый геометрией (полярной, продольной, поперечной) и оптической настройкой. В режиме «около экстинкции» сигнал фотоинтенсивности становится линейным по ρ_K и, следовательно, по компоненте $\mathbf{M}$, на которую настроена система.

Нормировка и калибровка

Применяется плоско-полевая коррекция $I_0(x, y)$ и нормировка $S = (I - I_0)/I_0$. Для установки абсолютного знака и масштаба выполняется дифференциальная съёмка на насыщении внешнего поля $+H_{\text{sat}}$ и $-H_{\text{sat}}$ с построением ΔS, что фиксирует sign и оценивает $\|\mathbf{b}\|$ при известной M_{sat}. В области малого смещения анализатора на угол δ от экстинкции интенсивность аппроксимируется

$$I_{\text{out}} \approx I_0 \left(\delta^2 + 2\,\delta\ \mathrm{Re}\,\rho_K + |\rho_K|^2\right),$$

и при $|\rho_K| \ll \delta \ll 1$ линейная модель по ρ_K оправдана.

Один кадр и два кадра

Для одного кадра определяется проекция $M_{\text{sens}} = (S - a)/\|\mathbf{b}\|$ и маска доменов с учётом двузначности направления $\pm\hat{\mathbf{b}}$. Для пары независимых кадров $\mathbf{b}_1, \mathbf{b}_2$ решается система $S - a = B\,\mathbf{M}$ с контролем обусловленности $\kappa(B)$; при необходимости вводится регуляризация Тихонова.

Подавление шума и карта доверия

Структурный тензор на изображении используется для оценки локальной когерентности и маскировки доменных стенок (как зон интенсивных градиентов), а также для задания весов уверенности. Визуализация включает векторное поле $\mathbf{M}$ и карту доверия, легенду геометрии.

Валидация и метрики

Используются: угловая ошибка $\Delta\phi$ на тестовых объёмах и синтетике; метрики сегментации (IoU, F1); стабильность к дрейфу/шуму, проверка линейности при вариации δ; мониторинг $\kappa(B)$ и размера регуляризации.

5 Выводы

- Предложен воспроизводимый конвейер: нормировка → калибровка знака/масштаба → оценка проекции/вектора → маскирование стенок → визуализация и метрики.
- Реализуемость на Swift/Core Image/Accelerate подтверждена архитектурой модулей и набором метаданных для повторяемой съёмки и обработки.
- Введены критерии устойчивости и качества, позволяющие контролировать применимость линейной модели и корректность восстановления направления $\mathbf{M}$.

Список литературы

1. Smith J. Example Article // Some Journal, 2022.
2. Qiu Z. Q., Bader S. D. Surface magneto-optic Kerr effect // Journal of Magnetism and Magnetic Materials, 2000.
3. Hubert A., Schäfer R. Magnetic Domains. Springer, 1998.
4. Zvezdin A. K., Kotov V. A. Modern Magnetooptics and Magnetooptical Materials. Taylor & Francis, 1997.
5. Soldatov I. V., Schäfer R. Advanced MOKE magnetometry in wide-field Kerr-microscopy // J. Appl. Phys. 122, 153906 (2017).
6. Soldatov I. V., Schäfer R. Selective sensitivity in Kerr microscopy // Rev. Sci. Instrum. 88, 073701 (2017).
7. Soldatov I. V., Schäfer R. Advances in quantitative Kerr microscopy // Phys. Rev. B 95, 014426 (2017).
8. Schäfer R. Investigation of domains and dynamics of domain walls by the magneto-optical Kerr-effect. In: Handbook of Magnetism and Advanced Magnetic Materials (John Wiley & Sons, Ltd, 2007).
9. Kuch W., Schäfer R., Fischer P., Hillebrecht F. Magnetic Microscopy of Layered Structures. Springer, New York, 2015.

10. Schmidt F., Rave W., Hubert A. Enhancement of magneto-optical Domain observation by digital image-processing // IEEE Trans. Magn. 21, 1596 (1985).
11. Faraday M. On the magnetisation of light and the illumination of magnetic lines of force // Philos. Trans. R. Soc. London 136, 1 (1846).
12. Marko D., Soldatov I., Tekielak M., Schäfer R. Stray-field-induced Faraday contributions in wide-field Kerr microscopy and -magnetometry // J. Magn. Magn. Mater. 396, 9 (2015).
13. Rave W., Reichel P., Brendel H., Leicht M., McCord J., Hubert A. Progress in quantitative magnetic domain observation // IEEE Trans. Magn. 29, 2551 (1993).
14. Soldatov I., Zehner J., Leistner K., Kang T., Karnaushenko D., Schäfer R. Advanced, Kerr-microscopy-based MOKE magnetometry for the anisotropy characterisation of magnetic films // J. Magn. Magn. Mater. 529, 167889 (2021).
15. Urs N. O., Mozooni B., Mazalski P., et al. Advanced magneto-optical microscopy: Imaging from picoseconds to centimeters - imaging spin waves and temperature distributions (invited) // AIP Advances 6, 055605 (2016).
16. Schäfer R., Oppeneer P. M., Ognev A. V., et al. Analyzer-free, intensity-based, wide-field magneto-optical microscopy // Appl. Phys. Rev. 8, 031402 (2021).
17. Oppeneer P. M. Magneto-optical Kerr spectra. In: Handbook of Magnetic Materials, Vol. 13 (Elsevier, 2001), pp. 229–422.
18. Rave W., Schäfer R., Hubert A. Quantitative observation of magnetic domains with the magneto-optical Kerr effect // J. Magn. Magn. Mater. 65, 7 (1987).
19. McCord J., Hubert A. Normalized differential Kerr microscopy an advanced method for magnetic imaging // Phys. Status Solidi A 171, 555–62 (1999).
20. Wolf G., Hamrle J., Trudel S., et al. Quadratic magneto-optical Kerr effect in Co2MnSi // J. Appl. Phys. 110, 043904 (2011).
21. Jiménez E., Mikuszeit N., Cuñado J. L. F., et al. Vectorial Kerr magnetometer for simultaneous and quantitative measurements of the in-plane magnetization components // Rev. Sci. Instrum. 85, 053904 (2014).
22. Teixeira J. M., Lusche R., Ventura J., et al. Versatile, high sensitivity, and automatized angular dependent vectorial Kerr magnetometer for the analysis of nanostructured materials // Rev. Sci. Instrum. 82, 043902 (2011).
23. Wu T.-h., Fu H., Hajjar R. A., Suzuki T., Mansuripur M. Measurement of magnetic anisotropy constant for magneto-optical recording media: A comparison of several techniques // J. Appl. Phys. 73, 1368–1376 (1993).
24. Postava K., Jaffres H., Schuhl A., et al. Linear and quadratic magneto-optical measurements of the spin reorientation in epitaxial Fe films on MgO // J. Magn. Magn. Mater. 172, 199–208 (1997).
25. Kerr J. On rotation of the plane of polarisation by reflection from the pole of a magnet // Philos. Mag. 3, 321–343 (1877).

A Физическое приложение

A.1 Магнитооптический эффект Керра

Магнитооптический эффект Керра — изменение поляризации отражённого света от намагниченного материала; наблюдаются вращение плоскости поляризации и эллиптичность, чувствительные к намагниченности в зоне зондирования света. В линейном по намагниченности приближении комплексный керровский угол $\rho_K = \theta_K + i\epsilon_K$ при p-поляризованном падении аппроксимируется через элементы матрицы отражения как $\rho_K \approx r_{ps}/r_{pp}$ (для s-падения $\rho_K \approx r_{sp}/r_{ss}$); чувствительность определяется внедиагональными компонентами диэлектрического тензора, пропорциональными компонентам $\mathbf{M}$.

A.2 Геометрии и чувствительность

Рабочие геометрии МОКЭ и чувствительные компоненты $\mathbf{M}$:

- Полярная: чувствительна к внеплоскостной компоненте M_z; удобна при близком к нормальному падении.
- Продольная: чувствительна к проекции в плоскости образца вдоль плоскости падения $M_\parallel$.
- Поперечная: проявляется как изменение отражения $\Delta R/R \propto M_\perp$ к плоскости падения; вращения плоскости поляризации нет.

При анализаторе около экстинкции фотосигнал становится к первой степени линейной функцией ρ_K, что даёт линейную зависимость измеряемого сигнала от соответствующей проекции $\mathbf{M}$.

A.3 Магнитные домены и стенки

Ферромагнитная плёнка разбивается на домены — области квази-единой ориентации $\mathbf{M}$, разделённые доменными стенками с быстрым поворотом направления; МОКЭ-микроскопия визуализирует контраст доменов за счёт изменения ρ_K при смене компоненты $\mathbf{M}$, на которую настроена чувствительность.

A.4 Линейная модель сигнала и ограничения

Введём нормированный сигнал $S(x, y)$ после плоско-полевой калибровки:

$$S(x, y) = \frac{I(x, y) - I_0(x, y)}{I_0(x, y)} \approx a + \mathbf{b} \cdot \mathbf{M}(x, y). \tag{A.1}$$

Здесь $\mathbf{b}$ — вектор чувствительности, задаваемый геометрией и настройкой поляризатор/анализатор. Одно изображение в фиксированной геометрии определяет лишь проекцию $\mathbf{M}$ на $\mathbf{b}$ и несёт двузначность $\pm$ (на 180°) без дополнительной априорной информации (ориентации внешнего поля/калибровки).

Режим около экстинкции и линейность. Пусть анализатор смещён от экстинкции на малый угол δ. Тогда

$$I_{\text{out}} \approx I_0\Big(\delta^2 + 2\,\delta\,\mathrm{Re}\,\rho_K + |\rho_K|^2\Big), \tag{A.2}$$

и для $|\rho_K| \ll \delta \ll 1$ вклад $2\delta \operatorname{Re} \rho_K$ доминирует, обосновывая линейную аппроксимацию $S \simeq a + \mathbf{b} \cdot \mathbf{M}$. Практически следует контролировать δ в эксперименте и проверять линейность по серии малых δ.

B Математическое приложение

B.1 Постановка

По входному МОКЭ-изображению $I(x, y)$ и метаданным съёмки восстановить: при одном кадре — карту возможных направлений $\pm\hat{\mathbf{b}}$ и знак проекции M_{sens}; при двух независимых кадрах — вектор (M_x, M_y) в плоскости образца.

B.2 Нормировка и наблюдаемые

Оценивается плоское поле $I_0(x, y)$ и определяется

$$S(x, y) = \frac{I(x, y) - I_0(x, y)}{I_0(x, y)}. \tag{B.1}$$

Геометрия съёмки задаёт единичный $\hat{\mathbf{b}}$ и масштаб $\|\mathbf{b}\|$.

B.3 Линейная модель и один кадр

При фиксированной геометрии

$$S(x, y) = a + \mathbf{b} \cdot \mathbf{M}(x, y). \tag{B.2}$$

Оценка «чувствительной» проекции:

$$M_{\text{sens}}(x, y) = \frac{S(x, y) - a}{\|\mathbf{b}\|}, \qquad \text{направление: } \pm\,\hat{\mathbf{b}}. \tag{B.3}$$

Выводится карта знака и величины проекции с учётом двузначности.

B.4 Два независимых кадра

Для двух кадров с линейно независимыми $\mathbf{b}_1, \mathbf{b}_2$ (в плоскости образца):

$$\begin{bmatrix} S_1 - a_1 \\ S_2 - a_2 \end{bmatrix} = \underbrace{\begin{bmatrix} b_{1x} & b_{1y} \\ b_{2x} & b_{2y} \end{bmatrix}}_{B} \begin{bmatrix} M_x \\ M_y \end{bmatrix}. \tag{B.4}$$

Тогда $(M_x, M_y)^\top = B^{-1}(S - a)$ при $\det B \neq 0$. Для улучшения устойчивости при большой $\kappa(B)$ используется

$$\mathbf{M} = (B^\top B + \lambda I)^{-1} B^\top (S - a), \tag{B.5}$$

где $\lambda > 0$ — малая регуляризация.

B.5 Сегментация доменов

Трёхзначная маска:

$$D(x,y) = \begin{cases} +1, & M_{\text{sens}} > T, \\ -1, & M_{\text{sens}} < -T, \\ 0, & |M_{\text{sens}}| \le T, \end{cases} \tag{B.6}$$

где T — робастный порог (например, по MAD) после сглаживания и вычитания фона.

B.6 Оценка локальной ориентации: структурный тензор

Пусть I_x, I_y — градиенты. Структурный тензор

$$J = G_\rho * \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}. \tag{B.7}$$

Угол локальной ориентации

$$\phi = \tfrac{1}{2} \arctan 2\big(2J_{xy},\ J_{xx} - J_{yy}\big), \tag{B.8}$$

когерентность

$$c = \frac{\lambda_1 - \lambda_2}{\lambda_1 + \lambda_2}, \tag{B.9}$$

где $\lambda_1 \ge \lambda_2$ — собственные значения J. Значение c используется для подавления шумовых областей и маскировки доменных стенок.

B.7 Предобработка и устойчивость

Применяются: гауссово сглаживание $G_\sigma * I$, вычитание медленного фона, морфологические операции (open/close), адаптивный порог; доверие пикселя по $|S|$ и c. Эти шаги стабилизируют знак и границы доменов перед визуализацией.

B.8 Калибровка на насыщении и знак

Снимается пара кадров при $+H_{\text{sat}}$ и $-H_{\text{sat}}$ в неизменной геометрии:

$$\Delta S = S(+H_{\text{sat}}) - S(-H_{\text{sat}}) \approx 2\,\|\mathbf{b}\|\, M_{\text{sat}} \cdot \text{sign}\,. \tag{B.10}$$

Это задаёт абсолютный знак и позволяет оценить $\|\mathbf{b}\|$ при известном M_{sat}. Рекомендуется сохранять ΔS как эталон для серии.

B.9 Проверка независимости и ковариация

Линейная независимость $\mathbf{b}_1, \mathbf{b}_2$ контролируется по $|\det B|$ и $\kappa(B)$. При гауссовском шуме с ковариацией Σ ковариация оценки

$$\text{Cov}(\mathbf{M}) \approx (B^\top \Sigma^{-1} B)^{-1}. \tag{B.11}$$

Это используется для построения карты доверия и отбора пикселей.

B.10 Псевдокод конвейера

1. Стабилизация и flat-field: оценить I_0, получить $S = (I - I_0)/I_0$.
2. Калибровка: снять ΔS на $\pm H_{\text{sat}}$, установить знак и $\|\mathbf{b}\|$.
3. Один кадр: $M_{\text{sens}} = (S - a)/\|\mathbf{b}\|$, пороговая маска после сглаживания.
4. Два кадра: собрать S_1, S_2, проверить $\kappa(B)$, восстановить $\mathbf{M}$ с регуляризацией при необходимости.
5. Структурный тензор: посчитать c, замаскировать стенки и шумовые зоны.
6. Визуализация: карта знака/вектора, карта доверия, легенда геометрии.

C Программное приложение

Для практической реализации алгоритмов анализа магнитных доменов было разработано приложение для операционной системы macOS. Приложение написано на языке Swift с использованием фреймворка SwiftUI для построения пользовательского интерфейса.

C.1 Архитектура приложения

Проект построен по модульному принципу, разделяя логику обработки данных и пользовательский интерфейс. Структура проекта включает три основных компонента:

- **course4_programApp.swift** — точка входа в приложение.
- **ContentView.swift** — описание пользовательского интерфейса и взаимодействия с пользователем.
- **generateVectorDiagram.swift** — реализация алгоритмов обработки изображений и анализа доменов.

Полный листинг исходного кода приведен в разделе D.

C.2 Алгоритмы обработки и анализа

Ядром приложения является модуль `generateVectorDiagram.swift` (см. Листинг 3). Процесс обработки изображения состоит из нескольких этапов.

C.2.1 Предобработка и кэширование

При загрузке изображения оно преобразуется в матрицу яркости (Grayscale). Поскольку операции вычисления порогов и фильтрации могут требовать ресурсов, реализован механизм кэширования через класс `ImageAnalysisCache` (Листинг 3, строки 11–34). Функция `createAnalysisCache` (строки 152–222) выполняет первичную обработку, в ходе которой вычисляются и сохраняются:

- Матрица яркости пикселей (строки 175–191).
- Минимальное и максимальное значения яркости (строки 180–190).
- Базовые пороги сегментации, вычисленные автоматически (строки 208–221).

Яркость пикселя вычисляется по стандартной формуле преобразования RGB в Grayscale (строки 163–173):

$$Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

Это позволяет быстрее пересчитывать результаты при ручной корректировке параметров пользователем без повторного чтения данных изображения.

C.2.2 Автоматическая пороговая сегментация (Метод Оцу)

Для автоматического определения границ доменов реализован модифицированный двухуровневый метод Оцу (функция `otsuDualThreshold`, Листинг 3, строки 43–149). Алгоритм находит два оптимальных порога, разделяющих пиксели на три класса:

1. Тёмные области (домены одного направления).
2. Светлые области (домены противоположного направления).
3. Нейтральные области (переходные зоны, границы, шум).

Для повышения устойчивости к шуму перед вычислением гистограммы отбрасываются по 5% самых темных и самых светлых пикселей (trimming, строки 62–91). Оптимизация порогов выполняется путём максимизации межклассовой дисперсии (строки 115–142).

C.2.3 Сегментация и фильтрация доменов

Функция `analyzeDomains` (Листинг 3, строки 225–406) выполняет сегментацию изображения на основе порогов. Пользователь имеет возможность корректировать пороги относительно базовых значений Оцу с помощью коэффициентов (от 0.8 до 1.2), что реализовано через функцию `applySliderMultiplier` (строки 37–40). Применение порогов выполняется в строках 234–235.

Для выделения связных областей используется алгоритм поиска в ширину (BFS, строки 257–304). Пиксели группируются в домены, после чего применяется фильтрация по размеру: области, занимающие менее 1/5000 площади изображения, считаются шумом и отбрасываются (строки 307–308).

C.2.4 Построение векторной диаграммы

Для каждого выделенного домена вычисляется его центроид (строки 370–381). В зависимости от классификации домена (светлый или темный) ему приписывается условное направление вектора намагниченности (в текущей реализации — вверх или вниз, отображаемые красным и синим цветами соответственно, строки 386–398). Также алгоритм определяет пиксели границ доменов для их визуализации: граничными считаются пиксели, имеющие соседей из другого домена или находящиеся на краю изображения (строки 333–366).

C.3 Пользовательский интерфейс

Интерфейс приложения (файл `ContentView.swift`, Листинг 2) предоставляет следующие возможности:

- Загрузка изображений форматов PNG и JPEG (функция `openImageFile`, строки 233–246).
- Визуализация исходного изображения (строки 110–120).
- Панель настроек с ползунками для точной подстройки порогов сегментации «на лету» (строки 68–103).

- Режим наложения результатов анализа: отображение границ доменов (строки 123–141) и векторного поля (строки 143–156) поверх оригинального изображения.

Для эффективной отрисовки большого количества графических примитивов используются специализированные компоненты на базе `Canvas`: `ArrowFieldView` для отрисовки стрелок (строки 181–193) и `DomainBoundaryView` для отрисовки точек границ (строки 196–225), обеспечивающие высокую производительность.

D Листинг программного кода

D.1 Точка входа (course4_programApp.swift)

```
//
//  course4_programApp.swift
//  course4_program
//

import SwiftUI

@main
struct course4_programApp: App {
    var body: some Scene {
        WindowGroup {
            ContentView()
        }
    }
}
```

Листинг 1: Точка входа в приложение

D.2 Интерфейс пользователя (ContentView.swift)

```
import SwiftUI
import UniformTypeIdentifiers
import AppKit

struct ContentView: View {
    @State private var image: NSImage?
    @State private var vectorArrows: [Arrow] = []

    @State private var brightBoundaryPoints: [CGPoint] = []
    @State private var darkBoundaryPoints: [CGPoint] = []

    @State private var showVectorDiagram: Bool = false

    //                                                  (                +
                                Otsu)
    @State private var analysisCache: ImageAnalysisCache?

    //                  : 0..1,         0.5 =                   Otsu
    //        0:        0       .8,         1:        1       .2
    @State private var darkSlider: Double = 0.5
    @State private var brightSlider: Double = 0.5

    private let canvasSize: CGFloat = 400

    var body: some View {
        VStack(spacing: 16) {

            HStack(spacing: 12) {
                Button("                                        ") {
                    openImageFile { selectedImage in
                        self.image = selectedImage
                        self.vectorArrows = []
```

```
                        self.brightBoundaryPoints = []
                        self.darkBoundaryPoints = []
                        self.showVectorDiagram = false
                        self.analysisCache = nil
                        //

                        self.darkSlider = 0.5
                        self.brightSlider = 0.5
                    }
                }

                Button("                                        ") {
                    guard let img = image else { return }

                    //                        ,

                    if analysisCache == nil {
                        analysisCache = createAnalysisCache(from: img)
                    }

                    guard let cache = analysisCache else { return }

                    let vis = analyzeDomains(
                        cache: cache,
                        targetSize: canvasSize,
                        darkSlider: darkSlider,
                        brightSlider: brightSlider
                    )
                    self.vectorArrows = vis.arrows
                    self.brightBoundaryPoints = vis.brightBoundaryPoints
                    self.darkBoundaryPoints = vis.darkBoundaryPoints
                    self.showVectorDiagram = true
                }
                .disabled(image == nil)
            }
            .padding(.top, 16)

            //

            // 0 = base 0.8, 0.5 = base (Otsu), 1 = base 1.2
            VStack(spacing: 8) {
                HStack {
                    Text("                        :")
                        .frame(width: 120, alignment: .leading)
                    Text(" 0 .80")
                        .font(.caption)
                        .foregroundColor(.secondary)
                    Slider(value: $darkSlider, in: 0...1)
                        .frame(width: 180)
                    Text(" 1 .20")
                        .font(.caption)
                        .foregroundColor(.secondary)
                    Text(sliderMultiplierText(darkSlider))
                        .frame(width: 50, alignment: .trailing)
                        .font(.caption)
                }

                HStack {
```

```
                    Text("                    :")
                        .frame(width: 120, alignment: .leading)
                    Text(" 0 .80")
                        .font(.caption)
                        .foregroundColor(.secondary)
                    Slider(value: $brightSlider, in: 0...1)
                        .frame(width: 180)
                    Text(" 1 .20")
                        .font(.caption)
                        .foregroundColor(.secondary)
                    Text(sliderMultiplierText(brightSlider))
                        .frame(width: 50, alignment: .trailing)
                        .font(.caption)
                }
            }
            .padding(.horizontal)

            Divider()

            ScrollView {
                if let nsImage = image {
                    VStack(spacing: 24) {

                        VStack(spacing: 8) {
                            Text("                ")
                                .foregroundColor(.secondary)

                            Image(nsImage: nsImage)
                                .resizable()
                                .scaledToFit()
                                .frame(width: canvasSize, height: canvasSize)
                                .border(Color.gray)
                        }

                        if showVectorDiagram {

                            VStack(spacing: 8) {
                                Text("                            ")
                                    .foregroundColor(.secondary)

                                Image(nsImage: nsImage)
                                    .resizable()
                                    .scaledToFit()
                                    .frame(width: canvasSize, height: canvasSize)
                                    .overlay(
                                        DomainBoundaryView(
                                            brightPoints: brightBoundaryPoints,
                                            darkPoints: darkBoundaryPoints
                                        )
                                        .frame(width: canvasSize, height: canvasSize)
                                        .allowsHitTesting(false)
                                    )
                                    .border(Color.blue)
```

```
                            }

                            VStack(spacing: 8) {
                                Text("                            ")
                                    .foregroundColor(.secondary)

                                Image(nsImage: nsImage)
                                    .resizable()
                                    .scaledToFit()
                                    .frame(width: canvasSize, height: canvasSize)
                                    .overlay(
                                        ArrowFieldView(arrows: vectorArrows)
                                            .frame(width: canvasSize, height: canvasSize)
                                            .allowsHitTesting(false)
                                    )
                                    .border(Color.blue)
                            }
                        }
                    }
                    .frame(maxWidth: .infinity)
                    .padding(.bottom, 24)
                } else {
                    Text("                            ")
                        .foregroundColor(.secondary)
                        .frame(maxWidth: .infinity, maxHeight: .infinity)
                        .padding()
                }
            }
        }
        .frame(minWidth: 600, minHeight: 800)
        .padding()
    }
}

struct Arrow {
    var start: CGPoint
    var end: CGPoint
    var color: Color
}

struct ArrowFieldView: View {
    var arrows: [Arrow]

    var body: some View {
        Canvas { context, size in
            for arrow in arrows {
                var path = Path()
                path.move(to: arrow.start)
                path.addLine(to: arrow.end)
                context.stroke(path, with: .color(arrow.color), lineWidth: 2)
            }
        }
    }
```

```
}

struct DomainBoundaryView: View {
    var brightPoints: [CGPoint] //
    var darkPoints: [CGPoint]   //

    var body: some View {
        Canvas { context, size in
            let dotSize: CGFloat = 2.0

            for p in brightPoints {
                let rect = CGRect(
                    x: p.x - dotSize / 2,
                    y: p.y - dotSize / 2,
                    width: dotSize,
                    height: dotSize
                )
                context.fill(Path(ellipseIn: rect), with: .color(.green)
    )
            }

            for p in darkPoints {
                let rect = CGRect(
                    x: p.x - dotSize / 2,
                    y: p.y - dotSize / 2,
                    width: dotSize,
                    height: dotSize
                )
                context.fill(Path(ellipseIn: rect), with: .color(.red))
            }
        }
    }
}

///

func sliderMultiplierText(_ slider: Double) -> String {
    let multiplier = 0.8 + slider * 0.4
    return String(format: "%.2f", multiplier)
}

func openImageFile(completion: @escaping (NSImage?) -> Void) {
    let panel = NSOpenPanel()
    panel.allowedContentTypes = [.png, .jpeg]
    panel.allowsMultipleSelection = false
    panel.canChooseDirectories = false

    if panel.runModal() == .OK,
       let url = panel.url,
       let nsImage = NSImage(contentsOf: url) {
        completion(nsImage)
    } else {
        completion(nil)
    }
}
```

Листинг 2: Пользовательский интерфейс

D.3 Логика анализа (generateVectorDiagram.swift)

```
import SwiftUI
import AppKit

struct DomainVisualization {
    let arrows: [Arrow]
    let brightBoundaryPoints: [CGPoint]  //
    let darkBoundaryPoints: [CGPoint]    //
}

///
class ImageAnalysisCache {
    let brightness: [[Double]]
    let width: Int
    let height: Int
    let minB: Double
    let maxB: Double
    let range: Double

    //                                ,
                    Otsu (
                   )
    let baseDarkThreshold: Double
    let baseBrightThreshold: Double

    init(brightness: [[Double]], width: Int, height: Int, minB: Double, maxB: Double,
         baseDarkThreshold: Double, baseBrightThreshold: Double) {
        self.brightness = brightness
        self.width = width
        self.height = height
        self.minB = minB
        self.maxB = maxB
        self.range = maxB - minB
        self.baseDarkThreshold = baseDarkThreshold
        self.baseBrightThreshold = baseBrightThreshold
    }
}

///
func applySliderMultiplier(baseValue: Double, slider: Double) -> Double {
    let multiplier = 0.8 + slider * 0.4
    return baseValue * multiplier
}

///                                          Otsu
                                3
func otsuDualThreshold(brightness: [[Double]], width: Int, height: Int, trimPercent: Double = 0.05) -> (dark: Double, bright: Double) {
    let numBins = 256
    var histogram = [Int](repeating: 0, count: numBins)
    var totalPixels = 0
```

```
    //
    for y in 0..<height {
        for x in 0..<width {
            let br = brightness[y][x]
            let bin = min(numBins - 1, max(0, Int(br * Double(numBins - 1))))
            histogram[bin] += 1
            totalPixels += 1
        }
    }

    guard totalPixels > 0 else {
        return (dark: 0.33, bright: 0.66)
    }

    //
    let trimCount = Int(Double(totalPixels) * trimPercent)
    var lowBin = 0
    var highBin = numBins - 1

    var cumSum = 0
    for i in 0..<numBins {
        cumSum += histogram[i]
        if cumSum >= trimCount {
            lowBin = i
            break
        }
    }

    cumSum = 0
    for i in stride(from: numBins - 1, through: 0, by: -1) {
        cumSum += histogram[i]
        if cumSum >= trimCount {
            highBin = i
            break
        }
    }

    var trimmedHistogram = [Int](repeating: 0, count: numBins)
    var trimmedTotalPixels = 0

    for i in lowBin...highBin {
        trimmedHistogram[i] = histogram[i]
        trimmedTotalPixels += histogram[i]
    }

    guard trimmedTotalPixels > 0 else {
        return (dark: 0.33, bright: 0.66)
    }

    var prob = [Double](repeating: 0, count: numBins)
    var cumProb = [Double](repeating: 0, count: numBins)
    var cumMean = [Double](repeating: 0, count: numBins)

    for i in 0..<numBins {
        prob[i] = Double(trimmedHistogram[i]) / Double(
```

```
    trimmedTotalPixels)
    }

    cumProb[0] = prob[0]
    cumMean[0] = 0

    for i in 1..<numBins {
        cumProb[i] = cumProb[i - 1] + prob[i]
        cumMean[i] = cumMean[i - 1] + Double(i) * prob[i]
    }

    let totalMean = cumMean[numBins - 1]

    var maxVariance = -1.0
    var bestT1 = numBins / 3
    var bestT2 = 2 * numBins / 3

    let step = max(1, numBins / 64)

    for t1 in stride(from: 1, to: numBins - 2, by: step) {
        for t2 in stride(from: t1 + 1, to: numBins - 1, by: step) {
            let w0 = cumProb[t1]
            let w1 = cumProb[t2] - cumProb[t1]
            let w2 = 1.0 - cumProb[t2]

            if w0 <= 0 || w1 <= 0 || w2 <= 0 { continue }

            let mu0 = cumMean[t1] / w0
            let mu1 = (cumMean[t2] - cumMean[t1]) / w1
            let mu2 = (totalMean - cumMean[t2]) / w2

            let variance = w0 * (mu0 - totalMean) * (mu0 - totalMean) +
                           w1 * (mu1 - totalMean) * (mu1 - totalMean) +
                           w2 * (mu2 - totalMean) * (mu2 - totalMean)

            if variance > maxVariance {
                maxVariance = variance
                bestT1 = t1
                bestT2 = t2
            }
        }
    }

    let darkThreshold = Double(bestT1) / Double(numBins - 1)
    let brightThreshold = Double(bestT2) / Double(numBins - 1)

    return (dark: darkThreshold, bright: brightThreshold)
}

///
func createAnalysisCache(from image: NSImage) -> ImageAnalysisCache? {
    guard
        let tiffData = image.tiffRepresentation,
        let bitmap = NSBitmapImageRep(data: tiffData)
    else {
        return nil
    }

```

```
    let width = bitmap.pixelsWide
    let height = bitmap.pixelsHigh

    func brightnessAt(x: Int, y: Int) -> Double? {
        guard
            let color = bitmap.colorAt(x: x, y: y)?
                .usingColorSpace(NSColorSpace.deviceRGB)
        else { return nil }

        let r = Double(color.redComponent)
        let g = Double(color.greenComponent)
        let b = Double(color.blueComponent)
        return 0.299 * r + 0.587 * g + 0.114 * b
    }

    var brightness: [[Double]] = Array(
        repeating: Array(repeating: 0.0, count: width),
        count: height
    )

    var minB = 1.0
    var maxB = 0.0

    for y in 0..<height {
        for x in 0..<width {
            if let br = brightnessAt(x: x, y: y) {
                brightness[y][x] = br
                if br < minB { minB = br }
                if br > maxB { maxB = br }
            }
        }
    }

    guard maxB > minB else {
        return nil
    }

    let range = maxB - minB
    var normalizedBrightness: [[Double]] = Array(
        repeating: Array(repeating: 0.0, count: width),
        count: height
    )
    for y in 0..<height {
        for x in 0..<width {
            normalizedBrightness[y][x] = (brightness[y][x] - minB) /
range
        }
    }

    let thresholds = otsuDualThreshold(brightness: normalizedBrightness,
width: width, height: height)

    let baseDarkThreshold = minB + thresholds.dark * range
    let baseBrightThreshold = minB + thresholds.bright * range

    return ImageAnalysisCache(
        brightness: brightness,
        width: width,
```

```
        height: height,
        minB: minB,
        maxB: maxB,
        baseDarkThreshold: baseDarkThreshold,
        baseBrightThreshold: baseBrightThreshold
    )
}

///
func analyzeDomains(cache: ImageAnalysisCache,
                    targetSize: CGFloat = 400,
                    darkSlider: Double = 0.5,
                    brightSlider: Double = 0.5) -> DomainVisualization {

    let width = cache.width
    let height = cache.height
    let brightness = cache.brightness

    let tDark = applySliderMultiplier(baseValue: cache.baseDarkThreshold, slider: darkSlider)
    let tBright = applySliderMultiplier(baseValue: cache.baseBrightThreshold, slider: brightSlider)

    struct Domain {
        var points: [(Int, Int)]
        var isBright: Bool
        var index: Int
    }

    var labels = Array(
        repeating: Array(repeating: -1, count: width),
        count: height
    )

    var domains: [Domain] = []

    let neighbors = [
        (dx: 1,  dy: 0),
        (dx: -1, dy: 0),
        (dx: 0,  dy: 1),
        (dx: 0,  dy: -1)
    ]

    for y in 0..<height {
        for x in 0..<width {
            let br = brightness[y][x]
            let isBright = br >= tBright
            let isDark = br <= tDark

            if !(isBright || isDark) { continue }
            if labels[y][x] != -1 { continue }

            let classIsBright = isBright
            let labelIndex = domains.count

            var queue: [(Int, Int)] = []
            var pts: [(Int, Int)] = []

```

```
            labels[y][x] = labelIndex
            queue.append((x, y))

            while !queue.isEmpty {
                let (cx, cy) = queue.removeFirst()
                pts.append((cx, cy))

                for n in neighbors {
                    let nx = cx + n.dx
                    let ny = cy + n.dy

                    if nx < 0 || nx >= width || ny < 0 || ny >= height { continue }
                    if labels[ny][nx] != -1 { continue }

                    let nbr = brightness[ny][nx]
                    let nIsBright = nbr >= tBright
                    let nIsDark = nbr <= tDark

                    if classIsBright {
                        if !nIsBright { continue }
                    } else {
                        if !nIsDark { continue }
                    }

                    labels[ny][nx] = labelIndex
                    queue.append((nx, ny))
                }
            }

            domains.append(
                Domain(points: pts, isBright: classIsBright, index: labelIndex)
            )
        }
    }

    let minDomainPixels = max(50, (width * height) / 5000)
    let filteredDomains = domains.filter { $0.points.count >= minDomainPixels }

    let imageAspect = CGFloat(width) / CGFloat(height)
    var displayWidth: CGFloat
    var displayHeight: CGFloat
    var offsetX: CGFloat = 0
    var offsetY: CGFloat = 0

    if imageAspect > 1.0 {
        displayWidth = targetSize
        displayHeight = targetSize / imageAspect
        offsetY = (targetSize - displayHeight) / 2
    } else {
        displayHeight = targetSize
        displayWidth = targetSize * imageAspect
        offsetX = (targetSize - displayWidth) / 2
    }

    let scaleX = displayWidth / CGFloat(width)
```

```
    let scaleY = displayHeight / CGFloat(height)

    var arrows: [Arrow] = []
    var brightBoundaryPoints: [CGPoint] = []
    var darkBoundaryPoints: [CGPoint] = []

    for domain in filteredDomains {
        let labelIndex = domain.index

        for (x, y) in domain.points {
            var isBoundary = false

            for n in neighbors {
                let nx = x + n.dx
                let ny = y + n.dy

                if nx < 0 || nx >= width || ny < 0 || ny >= height {
                    isBoundary = true
                    break
                }

                if labels[ny][nx] != labelIndex {
                    isBoundary = true
                    break
                }
            }

            if isBoundary {
                let px = CGFloat(x) * scaleX + offsetX
                let py = CGFloat(y) * scaleY + offsetY
                let p = CGPoint(x: px, y: py)

                if domain.isBright {
                    brightBoundaryPoints.append(p)
                } else {
                    darkBoundaryPoints.append(p)
                }
            }
        }
    }

    let baseLength: CGFloat = 40

    for domain in filteredDomains {
        var sumX = 0.0
        var sumY = 0.0

        for (x, y) in domain.points {
            sumX += Double(x)
            sumY += Double(y)
        }

        let count = Double(domain.points.count)
        let cx = sumX / count
        let cy = sumY / count

        let sx = CGFloat(cx) * scaleX + offsetX
        let sy = CGFloat(cy) * scaleY + offsetY
```

```
        let dir: CGFloat = domain.isBright ? 1.0 : -1.0
        let ex = sx
        let ey = sy + dir * baseLength

        let color: Color = domain.isBright ? .red : .blue

        arrows.append(
            Arrow(
                start: CGPoint(x: sx, y: sy),
                end: CGPoint(x: ex, y: ey),
                color: color
            )
        )
    }

    return DomainVisualization(
        arrows: arrows,
        brightBoundaryPoints: brightBoundaryPoints,
        darkBoundaryPoints: darkBoundaryPoints
    )
}
```

Листинг 3: Алгоритмы обработки